

Como você já está vendo essa tela no GitHub, significa que o repositório remoto (lá na nuvem) já foi criado. Agora, você precisa criar o repositório **local** (no seu computador) e conectá-lo a esse remoto.



Como você está começando o projeto do zero no seu computador, você vai usar a **primeira opção** que o GitHub te deu: "**crie um novo repositório na linha de comando**".

Aqui está o passo a passo exato do que você deve fazer. Abra o terminal (ou Git Bash) na pasta onde você quer que o projeto fique no seu computador e digite os comandos abaixo, **um por um**, apertando Enter após cada linha:

Passo a Passo dos Comandos

Para ficar tudo certinho, você pode abrir o seu terminal, ir até o local onde costuma salvar seus projetos (como a pasta Documentos, por exemplo) e digitar estes comandos antes de rodar aqueles que passei na mensagem anterior:

1. Crie a pasta com esse nome exato:

```
mkdir S06-26-AB-Equipe-36-Web_App_Development
```

2. Entre na pasta que você acabou de criar:

```
cd S06-26-AB-Equipe-36-Web_App_Development
```

Pronto! Depois de dar esse cd, você já estará dentro da pasta correta e poderá rodar todos aqueles comandos do git init, git add, etc., com segurança.

3. Criar o arquivo README

Isso cria um arquivo de texto com o título do projeto. O edital de vocês exige um README, então começar por ele é perfeito.

```
echo "# S06-26-AB-Equipe-36-Web_App_Development" >> README.md
```

4. Iniciar o repositório local

Isso diz ao seu computador: "A partir de agora, rastreie as mudanças nesta pasta usando o Git".

```
git init
```

5. Adicionar o arquivo ao "palco" (stage)

Isso prepara o arquivo README que você acabou de criar para ser salvo na linha do tempo do projeto.

```
git add README.md
```

6. Salvar a versão (commit)

Isso cria o seu primeiro "ponto de salvamento" oficial no histórico do projeto.

```
git commit -m "primeiro commit"
```

7. Definir o nome da branch principal

Isso garante que a linha do tempo principal do seu projeto se chame main (o padrão moderno do GitHub).

```
git branch -M main
```

8. Conectar o Local com o Remoto

É aqui que a mágica acontece. Esse comando vincula a pasta do seu computador com aquele repositório que já está lá no GitHub.

```
git remote add origin
```

```
git@github.com:No-Country-simulation/S06-26-AB-Equipe-36-Web_App_Development.git
```

9. Enviar os arquivos para o GitHub

Isso pega o seu "primeiro commit" e manda lá para a nuvem. O -u garante que nas próximas vezes você só precise digitar git push.

```
git push -u origin main
```

O que acontece depois?

Assim que você dê o Enter no último comando (git push), se você for lá na página do GitHub e atualizar (F5), aquela tela de instruções vai sumir e você verá o seu arquivo README.md bonitinho lá. A partir desse momento, seu ambiente está configurado e você e sua equipe já podem começar a criar as pastas do Front-end e da API em Java!

Como clonar o repositório?

Para baixar o nosso projeto para o seu computador e começar a trabalhar, é só seguir estes passos no seu terminal (ou Git Bash):

1. Abra o terminal na pasta onde você quer salvar o projeto.
2. Faça o clone do repositório baixando tudo da nuvem:

```
git clone  
https://github.com/No-Country-simulation/S06-26-AB-Equipe-36-Web_App_Devel  
opment.git
```

3. Entre na pasta do projeto que acabou de ser criada:

```
cd S06-26-AB-Equipe-36-Web_App_Development
```

4. Crie a sua própria branch de trabalho e já mude para ela:

```
git checkout -b feat/nome-da-sua-tarefa
```

Pronto! A partir desse momento você já está na sua própria linha do tempo e pode programar sem medo de quebrar o projeto principal."

Você já sabe qual parte específica do projeto (front, back, dados) essa sua nova colega vai assumir primeiro para já definirmos o nome exato da primeira branch dela?

Como criar uma branch personalizada sua:

O exemplo que usei foi utilizando meu nome, mude para o nome que quiser. Se você quer criar uma branch chamada **leo**, execute:

```
git switch -c leo
```

Isso vai:

1. Criar a branch **leo**;
2. Trocar automaticamente para ela.

Se estiver usando uma versão mais antiga do Git, use:

```
git checkout -b leo
```

Depois, para enviar essa branch para o GitHub:

```
git push -u origin leo
```

Você pode verificar se ela foi criada com:

```
git branch
```

A saída será algo parecido com:

```
* leo  
main
```

O ***** indica que você está atualmente na branch **leo**. A partir daí, todos os seus commits serão feitos nessa branch até que você volte para a **main** com:

```
git switch main
```

ou

```
git checkout main
```

Exemplo de como ficou a minha:

```
C:\Users\Leofe\S06-26-AB-Equipe-36-Web_App_Development>git switch -c leo  
Switched to a new branch 'leo'  
  
C:\Users\Leofe\S06-26-AB-Equipe-36-Web_App_Development>git push -u origin leo  
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)  
remote:  
remote: Create a pull request for 'leo' on GitHub by visiting:  
remote:      https://github.com/No-Country-simulation/S06-26-AB-Equipe-36-Web_App_Development/pull/new/leo  
remote:  
To github.com:No-Country-simulation/S06-26-AB-Equipe-36-Web_App_Development.git  
* [new branch]      leo -> leo  
branch 'leo' set up to track 'origin/leo'.  
  
C:\Users\Leofe\S06-26-AB-Equipe-36-Web_App_Development>git branch  
develop  
* leo  
main
```

O que eu fiz até agora:

- Configurei o repositório:
- Fiz o passo a passo no docs com ajuda de IA
- Criação e divisão das branches
- Passo a passo para Clonar o repositório

main: É a vitrine intocável. Só entra código aqui quando estiver 100% testado e funcionando. É essa branch que vai estar conectada ao Render ou Railway para o deploy automático do MVP.

develop: É o laboratório de integração. Todo mundo envia o código pronto para cá primeiro. Aqui vocês testam se o Front-end conversou direito com a API em Java. Se tudo der certo na **develop**, vocês mandam para a **main**.

Branches de Funcionalidade (Features): Cada desenvolvedor cria a sua própria ramificação temporária a partir da **develop** na hora de programar. Quando a tarefa acaba, ela é enviada para a **develop** e a branch pode ser apagada.

Padrão de Nomenclatura: Usem prefixos para deixar claro o que está sendo feito. Exemplos: **feat/front-onboarding** (para nova tela), **feat/back-api-orientar** (para código Java), **fix/bug-login** (para correção de erros).